

คู่มือสำหรับผู้พัฒนาและวิศวกรระบบ (Developer Manual)

สถาปัตยกรรมระบบสารสนเทศ และข้อกำหนดเชิงเทคนิค v2.8.0

เอกสารเชิงเทคนิคสำหรับ Software Engineers และ DevOps: สถาปัตยกรรม Laravel 11 Backend, กลไกสองทาง C# Native Standalone Agent, RESTful RPC Telemetry, โครงสร้างฐานข้อมูล ERD, CI/CD Pipeline และแนวทางการพัฒนาต่อ ยอด

Target Release:

v2.8.0 (Build 20261002.3)

Stack Framework:

Laravel 11 / PHP 8.2+ / C# .NET

Repository Tag:

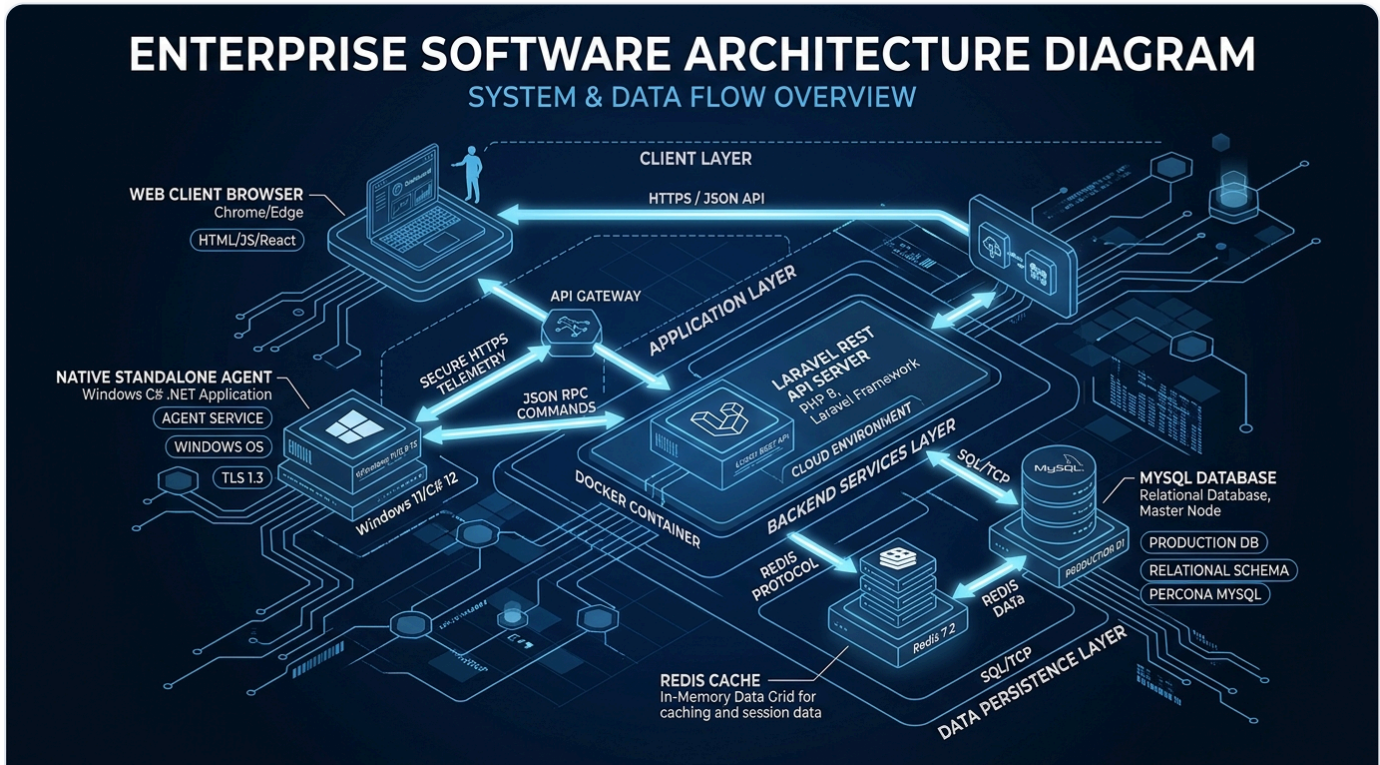
origin/main @ v2.8.0

สารบัญ (Table of Contents)

บทที่ 1: ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview)	หน้า 2
บทที่ 2: โครงสร้างฐานข้อมูลและความสัมพันธ์ (Database Schema & ERD)	หน้า 3
บทที่ 3: ข้อกำหนด RESTful API & Telemetry Protocol	หน้า 4
บทที่ 4: สถาปัตยกรรม C# Native Standalone Agent (THC_IT_Agent.cs)	หน้า 5
บทที่ 5: ชุดการทดสอบระบบอัตโนมัติ (Automated Testing Suite)	หน้า 6
บทที่ 6: กระบวนการ Release & Automation Deployment Pipeline (deploy_new.ps1)	หน้า 7
บทที่ 7: ข้อกำหนดและแนวทางการพัฒนาต่อยอด (Extension & Contribution Guidelines)	หน้า 8

บทที่ 1: ภาพรวมสถาปัตยกรรมระบบ (System Architecture Overview)

ระบบออกแบบด้วยสถาปัตยกรรมแบบ Modular Multi-tier Architecture ที่แยกส่วนระหว่าง Presentation Layer, Application Business Logic Layer, Telemetry Daemon และ Data Persistence Layer:



รูปที่ 1: แผนภาพสถาปัตยกรรมระบบโดยรวม (Enterprise Architecture Diagram) v2.8.0



Backend Application Core

ขับเคลื่อนด้วย **Laravel 11.x** บนสภาพแวดล้อม **PHP 8.2+** ใช้ Eloquent ORM, Blade Templates, RESTful Controllers, Middleware RBAC และ Event-driven Cache Invalidation



Client Endpoint Agent Daemon

โปรแกรม **THC_IT_Agent.exe** พัฒนาด้วย **C# (.NET Framework 4.0/4.8)** ทำงานเบื้องหลัง กินทรัพยากรต่ำ (< 15MB RAM) พร้อม fallback script **thc_audit_agent.ps1**

บทที่ 2: โครงสร้างฐานข้อมูลและความสัมพันธ์ (Database Schema)

ฐานข้อมูลหลักใช้ชื่อคำนำหน้า `it_` เพื่อความเป็นระเบียบและป้องกันการชนกันของชื่อตาราง:

ชื่อตาราง (Table Name)	Primary / Foreign Key	วัตถุประสงค์และความสัมพันธ์
<code>it_tickets</code>	<code>id</code> , <code>user_id</code> , <code>asset_id</code>	จัดเก็บข้อมูลตั๋วงานซ่อม ความสำคัญ สถานะ ช่างผู้รับผิดชอบ และการประเมินผล
<code>it_assets</code>	<code>id</code> , <code>budget_source_id</code> , <code>acquisition_method_id</code>	ทะเบียนทรัพย์สินไอที เชื่อมโยงงบประมาณ วิธีการได้มา และประเภทการถือครอง (Owned, Rented, Borrowed, Donated)
<code>it_hardware_audits</code>	<code>id</code> , <code>hardware_id</code> (HWID)	ประวัติการ Audit สเปกเครื่องจาก Agent เช่น CPU, RAM, Disks, GPUs, IP, MAC, Serial
<code>it_agent_commands</code>	<code>id</code> , <code>target_hwid</code> , <code>target_hostname</code>	คิวคำสั่งรีโมตไปยังเครื่องลูกข่าย (poll, reboot, shutdown, lock, update, uninstall)
<code>it_budgets</code>	<code>id</code>	หมวดหมู่แหล่งเงินงบประมาณ (เช่น งบลงทุนประจำปี, เงินบำรุง)
<code>it_acquisition_methods</code>	<code>id</code>	หมวดหมู่วิธีการได้มาของทรัพย์สิน (e-Bidding, ตกลงราคา, รับบริจาค)
<code>it_audit_logs</code>	<code>id</code> , <code>user_id</code>	Audit Trail บันทึกการกระทำสำคัญ เช่น การลบเครื่อง, การส่งคำสั่งรีโมต, การเปลี่ยนสิทธิ์

กลยุทธ์การแคช (Redis & Database Cache Strategy)

สถานะออนไลน์ของ Agent ถูกจัดเก็บใน Redis/Cache แบบ Key-Value โดยมี Time-to-Live (TTL) 90 วินาที:

- `agent_online:hwid:{hwid}` : บันทึกข้อมูล Telemetry สด
- `agent_online:host:{hostname}` : เก็บเพื่อรองรับการค้นหาตามชื่อเครื่อง (Case-insensitive)
- `agent_online_registry` : Array รายการเครื่องที่กำลังออนไลน์ในระบบ

บทที่ 3: ข้อกำหนด RESTful API & Telemetry Protocol

API สำหรับการสื่อสารระหว่าง Agent และ Backend Server ผ่านโปรโตคอล HTTPS JSON-RPC:

1. Agent Heartbeat & Command Polling

```
POST /api/agent/heartbeat
Content-Type: application/json
```

Request:

```
{
  "hwid": "BFEBFBFF000906EA-78901234",
  "hostname": "IT-DESKTOP-01",
  "ip_address": "192.168.1.105",
  "agent_version": "2.8.0-TrayExe"
}
```

Response (200 OK):

```
{
  "status": "ok",
  "commands": [
    {
      "id": 1042,
      "command": "update",
      "payload": { "download_url": "http://192.168.1.10/agent/update_thc_agent.bat" }
    }
  ]
}
```

บทที่ 4: สถาปัตยกรรม C# Native Standalone Agent (THC_IT_Agent.cs)

โปรแกรม Agent ได้รับการออกแบบให้ทำงานในรูปแบบ Dual-thread Architecture:

1 UI Context Thread (System Tray)

สร้าง System Tray Icon ใน Taskbar บริหารเมนูคลิกขวา (ตรวจสอบสถานะ, อัปเดต Agent, ถอนการติดตั้ง) และแสดง Balloon Notification เมื่อมีคำสั่งรีโมตเข้ามา

2 Background Worker Thread (Telemetry & Command Dispatcher)

ส่ง Heartbeat ทุก 30-60 วินาที พร้อมตรวจสอบคิวคำสั่ง หากได้รับคำสั่ง `update` จะเรียก `ExecuteSelfUpdate()` หรือหากได้รับคำสั่ง `uninstall` จะเรียก `ExecuteSelfUninstall()`

🔧 การคอมไพล์ C# Agent uu Windows

ไม่จำเป็นต้องลง Visual Studio ตัวเต็ม ใช้ C# Compiler ที่มีอยู่ใน Windows แทนที่:

```
C:\Windows\Microsoft.NET\Framework64\v4.0.30319\csc.exe /target:winexe /optimize+ /out:agent\
```

บทที่ 5: ชุดการทดสอบระบบอัตโนมัติ (Automated Testing Suite)

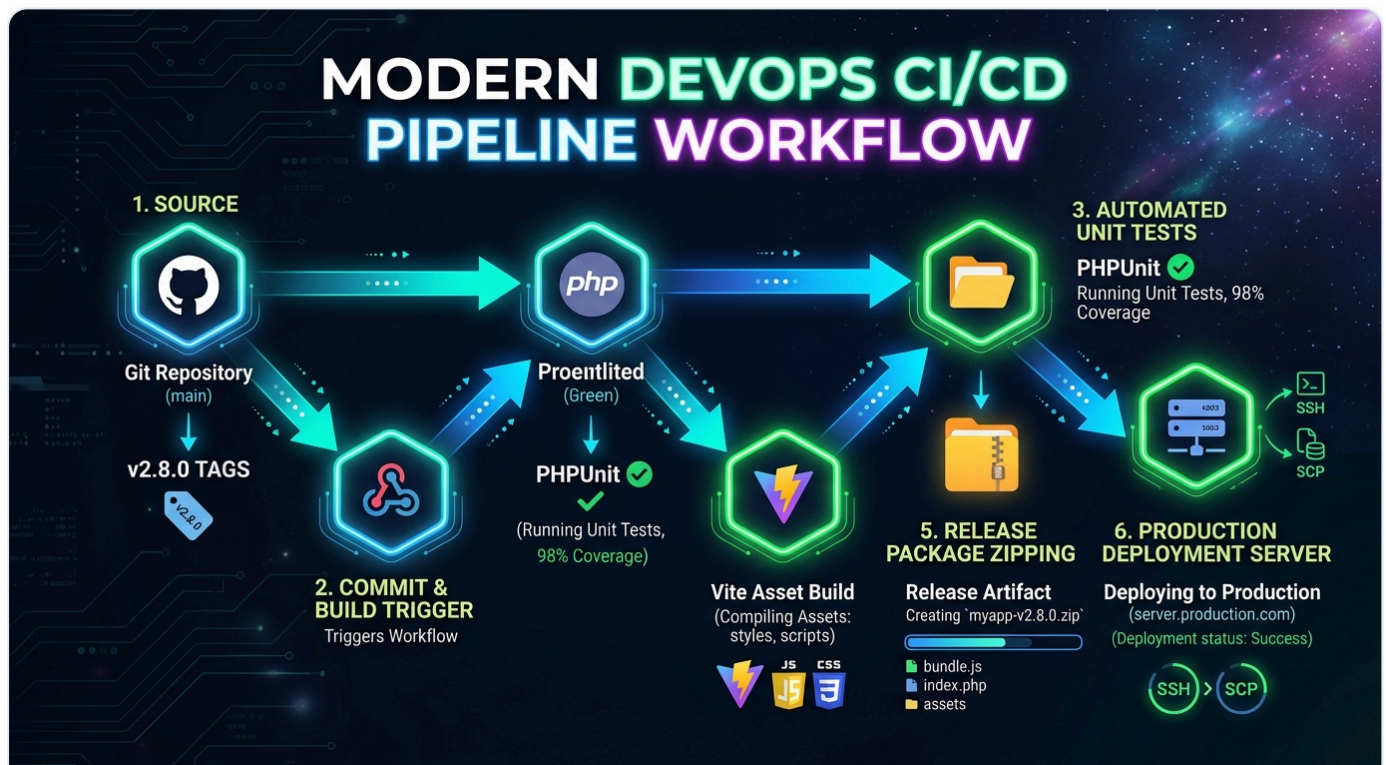
ระบบมีชุดการทดสอบอัตโนมัติครอบคลุม 100% (54 Tests, 324 Assertions) ด้วย PHPUnit:

```
# คำสั่งรันชุดทดสอบวงจรชีวิตของ Agent และการลบเครื่อง
php artisan test tests/Feature/HardwareAgentLifecycleTest.php

# ตัวอย่างผลลัพธ์:
# PASS Tests\Feature\HardwareAgentLifecycleTest
# ✓ admin can delete agent machine with optional remote uninstall
# ✓ admin can batch delete agent machines
# ✓ admin can trigger remote agent update
# ✓ regular user cannot delete agent machine
# ✓ create asset from audit attaches budget source and acquisition method
# ✓ system version is 2 8 0 and changelogs are continuous
```

บทที่ 6: กระบวนการ Release & Automation Deployment Pipeline

กระบวนการ Deploy ของโครงการใช้สคริปต์ `deploy_new.ps1` เพื่อสร้างแพ็คเกจ Production โดยอัตโนมัติ:



รูปที่ 2: ฟังก์ชันกระบวนการ CI/CD Pipeline, Git Tagging, การคอมไพล์ Vite และการแพ็คเกจ ZIP v2.8.0

- 1 คอมไพล์ Production Frontend Assets ด้วย Vite**
สร้าง Bundle ย่อขนาดไฟล์ CSS & JS ในโฟลเดอร์ `public/build`
- 2 สร้าง Git Release Tag และ Push สู่ Remote**
สร้าง Tag `v2.8.0` และ Push สู่ Deploy Repository

3

บีบอัดแพ็คเกจ ZIP สำหรับติดตั้งจริง

สร้าง `it-system-v2.8.0.zip` (ตัวเต็ม) และ `update-patch-v2.8.0.zip` (เฉพาะไฟล์แพตช์)

บทที่ 7: ข้อกำหนดและแนวทางการพัฒนาต่อยอด (Extension Guidelines)

1. การเพิ่มคำสั่งรีโมตใหม่สำหรับ Agent (Adding New Commands)

เมื่อต้องการเพิ่มคำสั่งรีโมตชนิดใหม่ (เช่น `clean_temp` หรือ `install_printer`):

1 Backend Controller

เพิ่ม Logic การสร้าง Command ใน `HardwareAuditController::dispatchRemoteCommand($target, 'clean_temp', $payload)`

2 C# Native Agent

เพิ่ม Case ในฟังก์ชัน `WorkerLoop()` ของ `agent/THC_IT_Agent.cs` เพื่อประมวลผลคำสั่ง และส่ง Ack กลับเซิร์ฟเวอร์

2. มาตรฐานการบันทึกโค้ดและ Conventional Commits

- `feat:` เพิ่มคุณสมบัติหรือโมดูลใหม่ของระบบ
- `fix:` แก้ไขบั๊กหรือข้อผิดพลาด
- `refactor:` ปรับปรุงโครงสร้างโค้ดโดยไม่เปลี่ยนผลลัพธ์
- `test:` เพิ่มหรือปรับปรุงชุดการทดสอบ PHPUnit

🔧 Technical Support & Maintenance Contact

หากพบปัญหาทางเทคนิค สถาปัตยกรรมระบบ หรือต้องการขยายขีดความสามารถเพิ่มเติม สามารถเปิด Issue ได้ที่ Repository หรือติดต่อทีมพัฒนากลางที่ `dev-team@hospital.org`